¿Cuáles son los tipos de Datos en Python?

Los nueve tipos de datos principales que se pueden usar en Python son los Booleans, los Numbers, las Strings, los Bytes y byte arrays, None, Listas, Tuplas, Sets y Diccionarios.

Un **boolean** se trata de un valor verdadero o falso. Por ejemplo, un boolean podría indicarnos si una entrada se ha publicado en un blog o no mostrando "falso" hasta que se publique. Con los booleans en Python, se debe poner en mayúsculas la T de True y la F de False.

Los tipos de expresiones de booleans que podemos usar son de comparación y operadores de lógica. Luego estas se pueden meter en condicionales, loops, etc. Estas serían las expresiones de comparación:

- "==" : Equal to
- "!=" : Not equal to
- ">" : Greater than
- "<" : Less than
- ">=" : Greater than or equal to
- "<=" : Less than or equal to

Un **número** es una parte compleja de Python. Sus diferentes opciones y bibliotecas compatibles con números pueden abarcar desde números enteros fijos (1, 2, 3… Se representan mediante int y long) hasta números decimales grandes, fracciones y números con coma flotante (float).

Las **strings** son una clase de cadena integrada de Python que pueden ser cualquier tipo de secuencia de bytes, desde un nombre hasta un documento HTML completo. Normalmente, las cadenas se cierran entre comillas simples o dobles.

Bytes y byte arrays: Los bytes son un tipo de dato reservado para el desarrollo complejo en Python, cuando se comienzan a realizar tareas como reorganizar secuencias de bytes o trabajar con imágenes a nivel de byte. Son secuencias de valores que se utilizan al trabajar con datos no textuales en formatos binarios específicos

"none" es un concepto que todo lenguaje de tipado dinámico tiene. Un ejemplo de "none" sería definir una variable, pero aún no se le quiere asignar ningún valor en particular. Es especialmente útil para comprobar si un valor sigue siendo "none" durante el desarrollo.

Las listas, tuplas, conjuntos y diccionarios se clasifican como estructuras de datos. Con ellas, se pueden gestionar colecciones en Python. Las colecciones son uno de los pilares fundamentales para crear aplicaciones web que implementan aprendizaje automático. A grandes rasgos y sin entrar en detalles, una lista sería similar a un array porque son ordenadas, mutables y permiten duplicados. Las tuplas y los conjuntos son similares a las listas, pero son inmutables. Los conjuntos son desordenados, mutables y

funcionan con elementos únicos. Los diccionarios, a diferencia de los tres primeros, permiten tener pares clave-valor.

¿Qué tipo de convención de nomenclatura deberíamos utilizar para las variables en Python?

A diferencia de JavaScript, cuya nomenclatura suele ser utilizar mayúsculas para separar las variables con más de una palabra, en Python se utiliza el under score o la barra baja cuando hay una variable con una colección de palabras o frases (por ejemplo: first_user_list). Otro asunto que vale la pena mencionar son las letras que nunca deben usarse como nombre de variable de un solo carácter. Estas incluyen: "L" minúscula, "O" mayúscula o "i" mayúscula, ya que en una práctica real pueden llegar a confundirse entre ellas y en el caso de la O puede parecer un cero.

La manera más efectiva de nombrar variables es aquella que contenga una descripción lo más auto referencial posible que nos ayude a entender qué contiene la variable sin necesidad de usar comentarios y ensuciar el código.

¿Qué es un Heredoc en Python?

En programación, los comentarios multilínea o "Heredocs" son bloques de texto que abarcan más de una línea y sirven para definir cadenas de texto largas manteniendo el formato original (saltos de línea, espacios, tabulaciones…). En los Heredocs el string no continúa al separar la línea, por lo que hay que añadir triple comilla al principio y al final (que pueden ser simples o dobles). Por ejemplo:

```
- content = """
  Nullam id dolor id nibh ultricies vehicula ut id elit. Nullam quis
  risus eget urna mollis ornare vel eu leo.
  Vestibulum id ligula porta felis euismod semper. Cum sociis natoque
  penatibus et magnis dis parturient montes, nascetur ridiculus mus.
  Cras justo odio, dapibus ac facilisis in
  Integer posuere erat a ante venenatis dapibus posuere velit aliquet.
  """
```

Para arreglar las líneas vacías que genera este string al imprimir el resultado se le puede poner un strip al final de las comillas, o poner una barra inversa \ al final de cada línea para escapar la línea nueva.

```
- .strip()
```

¿Qué es una interpolación de cadenas?

Una interpolación de cadenas (nombrada mediante f-strings) nos permite insertar variables o expresiones dentro de una cadena directamente. Por ejemplo, si tenemos la

variable de un nombre (name = "Andy") podemos definir esta variable dentro de otra que contenga un mensaje. Las variables se insertan mediante llaves y la interpolación de cadenas o f-string se define poniendo una f antes del mensaje.

- `Mensaje = f"Hola {name}, gracias por interesarte por nuestro producto."`
- `Print(mensaje)`
- `Salida: Hola Andy, gracias por interesarte por nuestro producto.`

¿Cuándo deberíamos usar comentarios en Python?

Para hacer un buen uso de los comentarios en Python hay que tener en cuenta que hay una gran diferencia entre un comentario que simplemente existe y uno que realmente aporta valor. A veces cambiar la nomenclatura y el cómo nombramos por ejemplo algunas variables puede hacer que el código se explique por sí solo sin necesidad de comentarios.

No es recomendable añadir comentarios al código para explicar su funcionalidad, ya que quizás en algún momento, el código puede cambiar y los comentarios acabarán no siendo verdad. El código debería documentarse a sí mismo y de forma que perdure en el tiempo.

Los comentarios pueden ser útiles para organizar las partes de nuestro código sin explicar demasiado su funcionalidad. Por ejemplo, para archivos de código grandes, como los archivos de estilos CSS. Podemos añadir /* Nav header styles */. En este caso se nos permite saber rápidamente dónde se puede encontrar un conjunto específico de estilos. Y como este tipo de comentarios no dan instrucciones ni describen el comportamiento, es mucho menos probable que queden obsoletos.

¿Cuáles son las diferencias entre aplicaciones monolíticas y de microservicios?

Las aplicaciones monolíticas tienen un código bastante más grande al estar todo integrado en el mismo lugar. Si los desarrolladores quieren cambiar algo deben acceder todos al mismo codebase. Las aplicaciones monolíticas, por lo tanto, tendrían la ventaja de ser más fáciles de testear y debuggear ya que todo el código estaría en el mismo lugar, más fáciles de desplegar y de desarrollar. Las aplicaciones monolíticas también suelen ser más rápidas que las de microservicios. No necesitan comunicarse con tantos servicios, por lo que no sufren el mismo retraso que algunas aplicaciones de microservicios.

Al estar todo integrado en el mismo lugar, en un mismo proceso, el código de las aplicaciones monolíticas termina siendo gigantesco y hacer algún cambio puede dar problemas de acoplamiento. Mientras, los microservicios dividen esos procesos. Se pueden crear varios servicios dentro de la aplicación y cada uno se puede probar y desarrollar individualmente. Otro beneficio de los microservicios es su escalabilidad ya

que se pueden escalar los elementos individualmente, además de ser más fácil todo el proceso de probarlos y testearlos para encontrar errores rápidamente. Si algo falla no perjudica al sistema entero por una única función, sino al apartado del servicio al que pertenece.